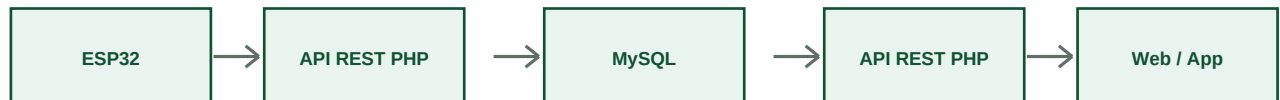


Manual de circuito y configuracion ESP32

Cableado sugerido, librerias, configuracion WiFi/API y codigo Arduino para el equipo electronico.

Todo pasa por HTTP JSON; nadie se conecta directo a MySQL excepto la API.



Documento de apoyo para montaje, configuracion y uso del proyecto. Usa rutas y archivos reales del repositorio para que el equipo pueda seguirlo sin adivinar.

1. Objetivo del montaje electronico

El ESP32 debe leer sensores fisicos, activar actuadores mediante etapas de potencia y comunicarse con la API REST PHP del proyecto. El ESP32 no se conecta directo a MySQL: envia y consulta datos por HTTP JSON.

Flujo esperado: ESP32 -> API REST PHP -> MySQL -> API REST PHP -> Web/App.

2. Material recomendado

- ESP32 DevKit o placa compatible.
- Sensor DHT11 para temperatura y humedad ambiente.
- Sensor YL-69 o equivalente para humedad de suelo analogica.
- Sensor BH1750 para intensidad de luz por I2C.
- Lector RFID RC522 y tarjetas/llaveros compatibles.
- Servo para puerta/acceso.
- Ventilador, bomba de agua y lampara LED.
- Modulos relay, MOSFET o drivers para controlar cargas. No conectar cargas directo a GPIO.
- Fuente externa para bomba, ventilador, lampara y servo cuando aplique.
- Protoboard, jumpers, resistencias, diodos flyback si se usan cargas inductivas y borneras si el montaje sera fijo.

3. Mapa de pines sugerido

Este mapa coincide con `esp32/firmware\_base/config.example.h`. Si cambian el cableado, tambien deben cambiar los `#define` del archivo `config.h`.

Componente	Conexion	Pin sugerido / nota
DHT11	VCC / GND / DATA	3.3V / GND / GPIO 4
YL-69 humedad suelo	VCC / GND / AO	3.3V / GND / GPIO 34
BH1750	VCC / GND / SDA / SCL	3.3V / GND / GPIO 21 / GPIO 22
RFID RC522	SDA, SCK, MOSI, MISO, RST	GPIO 5, 18, 23, 19, 27; VCC siempre 3.3V
Servo acceso	Senal / VCC / GND	GPIO 13 / fuente externa 5V / GND comun
Ventilador	Control relay/MOSFET	GPIO 25; alimentacion externa
Bomba	Control relay/MOSFET	GPIO 26; alimentacion externa
Lampara LED	Control relay/MOSFET	GPIO 33; alimentacion externa

4. Reglas electricas importantes

- No alimentar bomba, ventilador, lampara ni servo desde un GPIO del ESP32.
- Usar relay, MOSFET o driver adecuado a voltaje y corriente de la carga.
- Compartir GND entre ESP32 y fuente externa cuando el circuito de control lo requiera.
- El RC522 trabaja a 3.3V. No conectarlo a 5V.

- Si el relay es activo en LOW, ajustar `RELAY\_ON\_LEVEL` y `RELAY\_OFF\_LEVEL` en `config.h`.
- Antes de conectar cargas reales, probar cada salida con un LED o multímetro.

5. Preparar Arduino IDE

- 1 Instalar Arduino IDE.
- 2 Agregar soporte para placas ESP32 en Boards Manager.
- 3 Seleccionar la placa ESP32 usada y el puerto correcto.
- 4 Instalar librerías: ArduinoJson, DHT sensor library, Adafruit Unified Sensor, BH1750, MFRC522 y ESP32Servo.
- 5 Abrir `esp32/firmware\_base/firmware\_base.ino`.
- 6 Crear el archivo `config.h` en la misma carpeta.

6. Configurar API, WiFi y pines

Copiar `config.example.h` como `config.h`. Reemplazar SSID, contraseña y URL de API. Si el ESP32 y la PC están en la misma red, usar la IP LAN de la PC. Si se usa ngrok, usar la URL pública HTTPS.

Plantilla de config.h para pegar

```
#ifndef CONFIG_EXAMPLE_H
#define CONFIG_EXAMPLE_H

// Copia este archivo como config.h y reemplaza los placeholders locales.
// No subas config.h a Git.

// WiFi
#define WIFI_SSID "TU_SSID_WIFI"
#define WIFI_PASSWORD "TU_CONTRASEÑA_WIFI"

// API REST PHP. Ejemplos:
// - Local por red LAN: "http://192.168.1.50:8080/api"
// - Tunnel público: "https://TU-URL.ngrok-free.app/api"
#define API_BASE_URL "http://TU_HOST:8080/api"

// Demo: la UI oculta intervalo_lectura_seg, por eso el firmware usa 2 segundos fijos.
#define CICLO_DEMO_MS 2000UL

// DHT11
#define PIN_DHT 4
#define DHT_SENSOR_TYPE DHT11

// YL-69 humedad de suelo
#define PIN_HUMEDAD_SUELO_A0 34
#define HUMEDAD_SUELO_RAW_SECO 4095
#define HUMEDAD_SUELO_RAW_HUMEDO 1200

// BH1750 por I2C
#define PIN_I2C_SDA 21
#define PIN_I2C_SCL 22

// RFID RC522 por SPI
#define PIN_RFID_SS 5
#define PIN_RFID_SCK 18
#define PIN_RFID_MOSI 23
#define PIN_RFID_MISO 19
#define PIN_RFID_RST 27

// Servo de acceso
#define PIN_SERVO_ACCESO 13
#define SERVO_CERRADO_GRADOS 0
#define SERVO_ABIERTO_GRADOS 90
#define SERVO_TIEMPO_ABIERTO_MS 3000UL
#define SERVO_MIN_US 500
#define SERVO_MAX_US 2400

// Actuadores mediante relay, MOSFET o driver
#define PIN_RELAY_VENTILADOR 25
#define PIN_RELAY_BOMBA 26
#define PIN_RELAY_LAMPARA 33

// Ajusta estos niveles según el módulo usado.
// Muchos módulos relay son activos en LOW.
#define RELAY_ON_LEVEL HIGH
#define RELAY_OFF_LEVEL LOW

#endif
```

Ejemplos de API_BASE_URL

```
// PC y ESP32 en la misma red:
#define API_BASE_URL "http://192.168.1.50:8080/api"

// Prueba con tunel publico ngrok:
#define API_BASE_URL "https://TU-URL.ngrok-free.app/api"
```

7. Codigo principal para Arduino IDE

El codigo completo oficial esta en `esp32/firmware\_base/firmware\_base.ino`. Abrir ese archivo o copiar el anexo siguiente en un sketch de Arduino. Debe estar en la misma carpeta que `config.h`.

Nota de alcance: el firmware base envia lecturas, estados, comandos y RFID. La logica mas reciente de turnos por actuador esta probada en API/simulador; si se lleva a hardware real, se debe portar esa misma regla al firmware antes de entrega final.

Anexo A - firmware_base.ino completo

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>
#include <DHT.h>
#include <Wire.h>
#include <BH1750.h>
#include <SPI.h>
#include <MFRC522.h>
#include <ESP32Servo.h>

#if __has_include("config.h")
#include "config.h"
#else
#error "Copia config.example.h como config.h y configura tus credenciales locales."
#endif

struct ConfiguracionAutomatizacion {
    float temperaturaMaxC;
    float humedadSueloMinPct;
    float luzMinLux;
    bool ventilacionAutomatica;
    bool riegoAutomatico;
    bool iluminacionAutomatica;
    int duracionRiegoSeg;
};

struct LecturaSensores {
    float temperaturaC;
    float humedadAmbientePct;
    float humedadSueloPct;
    int humedadSueloRaw;
    float intensidadLuzLux;
};

WiFiClient clienteHttp;
WiFiClientSecure clienteHttps;
DHT dht(PIN_DHT, DHT_SENSOR_TYPE);
BH1750 bh1750;
MFRC522 rfid(PIN_RFID_SS, PIN_RFID_RST);
Servo servoAcceso;

ConfiguracionAutomatizacion configuracion = {
    30.0,
    35.0,
    500.0,
    true,
    true,
    true,
    5
};

int estadoVentilador = 0;
int estadoBomba = 0;
int estadoLampara = 0;
int estadoServoAcceso = 0;
int ultimaLecturaId = -1;

unsigned long ultimaEjecucion = 0;
unsigned long bombaEncendidaHasta = 0;
unsigned long servoAbiertoHasta = 0;

void conectarWiFi();
String requestApi(const String& metodo, const String& endpoint, const String& payload);
bool obtenerConfiguracion();
LecturaSensores leerSensores();
int enviarLectura(const LecturaSensores& lectura);
void aplicarAutomatizacion(const LecturaSensores& lectura);
bool procesarComandosPendientes();
void actualizarActuadores();
void registrarEvento(const String& actuador, int estadoAnterior, int estadoNuevo, const String& motivo);
```

```

void registrarAccesoRFID();

bool actuadorPermitido(const String& actuador);
int obtenerEstadoActuador(const String& actuador);
void asignarEstadoActuador(const String& actuador, int estado);
void cambiarActuadorAutomatico(const String& actuador, int estadoNuevo, const String& motivo, const String&
descripcion);
void marcarComando(int comandoId, const String& estadoComando, const String& respuesta);
void aplicarSalidasFisicas();
void actualizarServoAcceso();
String uidTarjetaActual();

void setup() {
  Serial.begin(115200);
  delay(500);

  pinMode(PIN_RELAY_VENTILADOR, OUTPUT);
  pinMode(PIN_RELAY_BOMBA, OUTPUT);
  pinMode(PIN_RELAY_LAMPARA, OUTPUT);
  aplicarSalidasFisicas();

  dht.begin();
  Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
  bh1750.begin(BH1750::CONTINUOUS_HIGH_RES_MODE);

  SPI.begin(PIN_RFID_SCK, PIN_RFID_MISO, PIN_RFID_MOSI, PIN_RFID_SS);
  rfid.PCD_Init();

  servoAcceso.setPeriodHertz(50);
  servoAcceso.attach(PIN_SERVO_ACCESO, SERVO_MIN_US, SERVO_MAX_US);
  servoAcceso.write(SERVO_CERRADO_GRADOS);

  conectarWiFi();
  obtenerConfiguracion();
  actualizarActuadores();
}

void loop() {
  registrarAccesoRFID();
  actualizarServoAcceso();

  if (millis() - ultimaEjecucion < CICLO_DEMO_MS) {
    return;
  }

  ultimaEjecucion = millis();

  Serial.println("=====");
  Serial.println("Ciclo ESP32 automatico iniciado");

  obtenerConfiguracion();
  LecturaSensores lectura = leerSensores();
  ultimaLecturaId = enviarLectura(lectura);

  aplicarAutomatizacion(lectura);
  actualizarActuadores();

  bool huboComandosManuales = procesarComandosPendientes();

  if (huboComandosManuales) {
    actualizarActuadores();
  }

  Serial.print("Estado final: ventilador=");
  Serial.print(estadoVentilador);
  Serial.print(", bomba=");
  Serial.print(estadoBomba);
  Serial.print(", lampara=");
  Serial.print(estadoLampara);
  Serial.print(", servo_acceso=");
  Serial.println(estadoServoAcceso);
}

void conectarWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.print("Conectando a WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.print("WiFi conectado. IP: ");
  Serial.println(WiFi.localIP());
}

String requestApi(const String& metodo, const String& endpoint, const String& payload) {
  if (WiFi.status() != WL_CONNECTED) {
    conectarWiFi();
  }

  HTTPClient http;
  String url = String(API_BASE_URL) + endpoint;

  if (url.startsWith("https://")) {
    // Demo con ngrok. En produccion se debe validar el certificado.
    clienteHttps.setInsecure();
  }
}

```

```

    http.begin(clienteHttps, url);
} else {
    http.begin(clienteHttp, url);
}

http.addHeader("Accept", "application/json");
http.addHeader("Content-Type", "application/json");

int codigoHttp = -1;

if (metodo == "GET") {
    codigoHttp = http.GET();
} else if (metodo == "POST") {
    codigoHttp = http.POST(payload);
} else if (metodo == "PUT") {
    codigoHttp = http.PUT(payload);
}

String respuesta = http.getString();

Serial.print(metodo);
Serial.print(" ");
Serial.print(endpoint);
Serial.print(" -> HTTP ");
Serial.println(codigoHttp);

if (codigoHttp < 200 || codigoHttp >= 300) {
    Serial.println(respuesta);
    http.end();
    return "";
}

http.end();
return respuesta;
}

bool obtenerConfiguracion() {
    String respuesta = requestApi("GET", "/configuracion.php", "");

    if (respuesta.length() == 0) {
        Serial.println("No se pudo obtener configuracion. Se conserva la ultima configuracion valida.");
        return false;
    }

    StaticJsonDocument<1536> doc;
    DeserializationError error = deserializeJson(doc, respuesta);

    if (error || !doc["ok"].as<bool>()) {
        Serial.println("Respuesta de configuracion no valida.");
        return false;
    }

    JsonObject cfg = doc["configuracion"];
    configuracion.temperaturaMaxC = cfg["temperatura_max_c"] | configuracion.temperaturaMaxC;
    configuracion.humedadSueloMinPct = cfg["humedad_suelo_min_pct"] | configuracion.humedadSueloMinPct;
    configuracion.luzMinLux = cfg["luz_min_lux"] | configuracion.luzMinLux;
    configuracion.ventilacionAutomatica = (cfg["ventilacion_automatica"] | 0) == 1;
    configuracion.riegoAutomatico = (cfg["riego_automatico"] | 0) == 1;
    configuracion.iluminacionAutomatica = (cfg["iluminacion_automatica"] | 0) == 1;
    configuracion.duracionRiegoSeg = cfg["duracion_riego_seg"] | configuracion.duracionRiegoSeg;

    Serial.print("Configuracion remota: temp_max=");
    Serial.print(configuracion.temperaturaMaxC);
    Serial.print(" C, humedad_min=");
    Serial.print(configuracion.humedadSueloMinPct);
    Serial.print(" %, luz_min=");
    Serial.print(configuracion.luzMinLux);
    Serial.print(" lux, ventilacion=");
    Serial.print(configuracion.ventilacionAutomatica);
    Serial.print(" , riego=");
    Serial.print(configuracion.riegoAutomatico);
    Serial.print(" , iluminacion=");
    Serial.print(configuracion.iluminacionAutomatica);
    Serial.print(" , duracion_riego=");
    Serial.print(configuracion.duracionRiegoSeg);
    Serial.println(" s");

    return true;
}

LecturaSensores leerSensores() {
    LecturaSensores lectura;

    float temperatura = dht.readTemperature();
    float humedadAmbiente = dht.readHumidity();

    if (isnan(temperatura)) {
        temperatura = 28.5 + sin(millis() / 6000.0);
    }

    if (isnan(humedadAmbiente)) {
        humedadAmbiente = 62.0 + sin(millis() / 7000.0);
    }

    int humedadRaw = analogRead(PIN_HUMEDAD_SUELO_A0);
    int humedadPct = map(humedadRaw, HUMEDAD_SUELO_RAW_SECO, HUMEDAD_SUELO_RAW_HUMEDO, 0, 100);
    humedadPct = constrain(humedadPct, 0, 100);

    float lux = bh1750.readLightLevel();

```

```

    if (lux < 0) {
        lux = 780.0 + sin(millis() / 8000.0) * 30.0;
    }

    lectura.temperaturaC = temperatura;
    lectura.humedadAmbientePct = humedadAmbiente;
    lectura.humedadSueloPct = humedadPct;
    lectura.humedadSueloRaw = humedadRaw;
    lectura.intensidadLuzLux = lux;

    Serial.print("Lectura sensores: temp=");
    Serial.print(lectura.temperaturaC);
    Serial.print(" C, humedad_ambiente=");
    Serial.print(lectura.humedadAmbientePct);
    Serial.print(" %, humedad_suelo=");
    Serial.print(lectura.humedadSueloPct);
    Serial.print(" %, raw=");
    Serial.print(lectura.humedadSueloRaw);
    Serial.print(" ", luz=");
    Serial.print(lectura.intensidadLuzLux);
    Serial.println(" lux");

    return lectura;
}

int enviarLectura(const LecturaSensores& lectura) {
    StaticJsonDocument<512> doc;
    doc["temperatura_c"] = lectura.temperaturaC;
    doc["humedad_ambiente_pct"] = lectura.humedadAmbientePct;
    doc["humedad_suelo_pct"] = lectura.humedadSueloPct;
    doc["humedad_suelo_raw"] = lectura.humedadSueloRaw;
    doc["intensidad_luz_lux"] = lectura.intensidadLuzLux;

    String payload;
    serializeJson(doc, payload);

    String respuesta = requestApi("POST", "/lecturas.php", payload);

    if (respuesta.length() == 0) {
        return -1;
    }

    StaticJsonDocument<512> respuestaDoc;
    DeserializationError error = deserializeJson(respuestaDoc, respuesta);

    if (error || !respuestaDoc["ok"].as<bool>()) {
        return -1;
    }

    int lecturaId = respuestaDoc["id"] | -1;
    Serial.print("Lectura enviada con id=");
    Serial.println(lecturaId);
    return lecturaId;
}

void aplicarAutomatizacion(const LecturaSensores& lectura) {
    Serial.println("Decisiones automaticas:");

    if (configuracion.ventilacionAutomatica) {
        if (lectura.temperaturaC > configuracion.temperaturaMaxC) {
            cambiarActuadorAutomatico("ventilador", 1, "temperatura_alta", "temperatura alta / ventilador encendido");
        } else {
            cambiarActuadorAutomatico("ventilador", 0, "temperatura_normal", "temperatura normal / ventilador apagado");
        }
    } else {
        Serial.println(" - ventilacion automatica desactivada / ventilador sin cambios");
    }

    if (configuracion.riegoAutomatico) {
        if (lectura.humedadSueloPct < configuracion.humedadSueloMinPct) {
            if (estadoBomba == 0) {
                bombaEncendidaHasta = millis() + ((unsigned long)configuracion.duracionRiegoSeg * 1000UL);
                cambiarActuadorAutomatico("bomba", 1, "suelo_seco", "humedad baja / bomba encendida");
            } else if (configuracion.duracionRiegoSeg > 0 && millis() >= bombaEncendidaHasta) {
                cambiarActuadorAutomatico("bomba", 0, "suelo_humedo", "duracion de riego cumplida / bomba apagada");
            } else {
                Serial.println(" - humedad baja / bomba continua encendida dentro de la duracion configurada");
            }
        } else {
            bombaEncendidaHasta = 0;
            cambiarActuadorAutomatico("bomba", 0, "suelo_humedo", "humedad suficiente / bomba apagada");
        }
    } else {
        Serial.println(" - riego automatico desactivado / bomba sin cambios");
    }

    if (configuracion.iluminacionAutomatica) {
        if (lectura.intensidadLuzLux < configuracion.luzMinLux) {
            cambiarActuadorAutomatico("lampara", 1, "luz_baja", "luz baja / lampara encendida");
        } else {
            cambiarActuadorAutomatico("lampara", 0, "luz_suficiente", "luz suficiente / lampara apagada");
        }
    } else {
        Serial.println(" - iluminacion automatica desactivada / lampara sin cambios");
    }
}

void cambiarActuadorAutomatico(const String& actuador, int estadoNuevo, const String& motivo, const String&

```

```

descripcion) {
    int estadoAnterior = obtenerEstadoActuador(actuador);

    if (estadoAnterior == estadoNuevo) {
        Serial.print(" - ");
        Serial.print(descripcion);
        Serial.println(" (sin cambio)");
        return;
    }

    asignarEstadoActuador(actuador, estadoNuevo);
    registrarEvento(actuador, estadoAnterior, estadoNuevo, motivo);

    Serial.print(" - ");
    Serial.print(descripcion);
    Serial.print(" (cambio ");
    Serial.print(estadoAnterior);
    Serial.print(" -> ");
    Serial.print(estadoNuevo);
    Serial.println(")");
}

bool procesarComandosPendientes() {
    String respuesta = requestApi("GET", "/comandos.php?estado=pendiente&limite=50", "");

    if (respuesta.length() == 0) {
        return false;
    }

    StaticJsonDocument<4096> doc;
    DeserializationError error = deserializeJson(doc, respuesta);

    if (error || !doc["ok"].as<bool>()) {
        Serial.println("Respuesta de comandos no valida.");
        return false;
    }

    JsonArray comandos = doc["comandos"].as<JsonArray>();
    int ejecutados = 0;
    int fallidos = 0;

    for (JsonObject comando : comandos) {
        int comandoId = comando["id"] | -1;
        String actuador = comando["actuador"] | "";
        int estadoSolicitado = comando["estado_solicitado"] | -1;

        if (comandoId <= 0 || estadoSolicitado < 0) {
            continue;
        }

        if (!actuadorPermitido(actuador)) {
            marcarComando(comandoId, "fallido", "servo_acceso no se controla desde app");
            fallidos++;
            continue;
        }

        int estadoAnterior = obtenerEstadoActuador(actuador);
        asignarEstadoActuador(actuador, estadoSolicitado);
        registrarEvento(actuador, estadoAnterior, estadoSolicitado, "comando_manual");
        marcarComando(comandoId, "ejecutado", "Comando ejecutado por ESP32");
        ejecutados++;
    }

    Serial.print("Comandos manuales procesados: ejecutados=");
    Serial.print(ejecutados);
    Serial.print(", fallidos=");
    Serial.println(fallidos);

    return ejecutados > 0 || fallidos > 0;
}

void actualizarActuadores() {
    aplicarSalidasFisicas();

    StaticJsonDocument<512> doc;
    doc["ventilador"] = estadoVentilador;
    doc["bomba"] = estadoBomba;
    doc["lampara"] = estadoLampara;
    doc["servo_acceso"] = estadoServoAcceso;
    doc["modo_control"] = "automatico";
    doc["origen"] = "esp32";

    if (ultimaLecturaId > 0) {
        doc["lectura_id"] = ultimaLecturaId;
    }

    String payload;
    serializeJson(doc, payload);
    requestApi("POST", "/actuadores.php", payload);
}

void registrarEvento(const String& actuador, int estadoAnterior, int estadoNuevo, const String& motivo) {
    StaticJsonDocument<512> doc;
    doc["actuador"] = actuador;
    doc["estado_anterior"] = estadoAnterior;
    doc["estado_nuevo"] = estadoNuevo;
    doc["motivo"] = motivo;

    if (ultimaLecturaId > 0) {
        doc["lectura_id"] = ultimaLecturaId;
    }
}

```



```

    } else {
        doc["lectura_id"] = nullptr;
    }

    doc["acceso_rfid_id"] = nullptr;

    String payload;
    serializeJson(doc, payload);
    requestApi("POST", "/eventos.php", payload);
}

void registrarAccesoRFID() {
    if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
        return;
    }

    String uid = uidTarjetaActual();

    StaticJsonDocument<256> doc;
    doc["uid"] = uid;
    doc["servo_abierto"] = 1;

    String payload;
    serializeJson(doc, payload);

    String respuesta = requestApi("POST", "/accesos.php", payload);

    if (respuesta.length() > 0) {
        StaticJsonDocument<512> respuestaDoc;

        if (!deserializeJson(respuestaDoc, respuesta) && respuestaDoc["autorizado"].as<bool>()) {
            estadoServoAcceso = 1;
            servoAcceso.write(SERVO_ABIERTO_GRADOS);
            servoAbiertoHasta = millis() + SERVO_TIEMPO_ABIERTO_MS;
            actualizarActuadores();
        }
    }

    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();
}

bool actuadorPermitido(const String& actuador) {
    return actuador == "ventilador" || actuador == "bomba" || actuador == "lampara";
}

int obtenerEstadoActuador(const String& actuador) {
    if (actuador == "ventilador") {
        return estadoVentilador;
    }

    if (actuador == "bomba") {
        return estadoBomba;
    }

    if (actuador == "lampara") {
        return estadoLampara;
    }

    return 0;
}

void asignarEstadoActuador(const String& actuador, int estado) {
    if (actuador == "ventilador") {
        estadoVentilador = estado;
    } else if (actuador == "bomba") {
        estadoBomba = estado;
    } else if (actuador == "lampara") {
        estadoLampara = estado;
    }
}

// servo_acceso se controla por RFID, no por comandos manuales.
}

void marcarComando(int comandoId, const String& estadoComando, const String& respuesta) {
    StaticJsonDocument<512> doc;
    doc["id"] = comandoId;
    doc["estado_comando"] = estadoComando;
    doc["respuesta_esp32"] = respuesta;

    String payload;
    serializeJson(doc, payload);
    requestApi("PUT", "/comandos.php", payload);
}

void aplicarSalidasFisicas() {
    digitalWrite(PIN_RELAY_VENTILADOR, estadoVentilador ? RELAY_ON_LEVEL : RELAY_OFF_LEVEL);
    digitalWrite(PIN_RELAY_BOMBA, estadoBomba ? RELAY_ON_LEVEL : RELAY_OFF_LEVEL);
    digitalWrite(PIN_RELAY_LAMPARA, estadoLampara ? RELAY_ON_LEVEL : RELAY_OFF_LEVEL);
}

void actualizarServoAcceso() {
    if (estadoServoAcceso == 1 && servoAbiertoHasta > 0 && millis() >= servoAbiertoHasta) {
        estadoServoAcceso = 0;
        servoAbiertoHasta = 0;
        servoAcceso.write(SERVO_CERRADO_GRADOS);
        actualizarActuadores();
    }
}

```

```
String uidTarjetaActual() {
  String uid = "";

  for (byte i = 0; i < rfid.uid.size; i++) {
    if (rfid.uid.uidByte[i] < 0x10) {
      uid += "0";
    }

    uid += String(rfid.uid.uidByte[i], HEX);
  }

  uid.toUpperCase();
  return uid;
}
```

8. Probar comunicacion con la API

- 1 Levantar Docker con `docker compose up -d --build`.
- 2 Abrir `http://localhost:8080/api/status.php` y confirmar `ok=true`.
- 3 Si se usa red local, obtener la IP de la PC con `ipconfig` y ponerla en `API\_BASE\_URL`.
- 4 Si se usa ngrok, ejecutar `ngrok http 8080` y pegar la URL HTTPS en `API\_BASE\_URL`.
- 5 Cargar el sketch al ESP32 y abrir Monitor Serial a 115200.
- 6 Confirmar mensajes de WiFi conectado, configuracion remota, lecturas y respuestas HTTP 2xx.

9. Checklist final antes de entregar circuito

- DHT11 entrega temperatura y humedad ambiente.
- YL-69 entrega valor raw y porcentaje de humedad de suelo razonable.
- BH1750 entrega lux mayor o igual a cero.
- RC522 detecta UID y el sistema registra acceso RFID.
- Servo abre y cierra sin reiniciar el ESP32.
- Ventilador, bomba y lampara cambian por relay/MOSFET sin alimentar cargas desde GPIO.
- La pagina web muestra lecturas y estados despues de cargar el firmware.
- No hay cables sueltos ni alimentaciones cruzadas peligrosas.